Apache Airflow

Beginner-Friendly Data Engineering & ETL/ELT Reference

Concepts • Architecture • DAGs • Scheduling • BashOperator • PythonOperator

1. Learning Objectives

After studying this document, a beginner should be able to:

- Explain ETL and ELT and understand where Airflow fits.
- Explain why workflow orchestration is required in data engineering.
- Describe the major Apache Airflow architecture components.
- Understand DAG, Task, Operator, Scheduler, Executor, Worker, Webserver, and Metadata Database.
- Create a simple DAG and define task dependencies.
- Use BashOperator to execute shell commands.
- Use PythonOperator to execute Python functions.
- Understand scheduling, retries, task states, logs, and DAG runs.
- Relate an Airflow DAG to a real-world ETL/ELT pipeline.

2. What is Apache Airflow?

Apache Airflow is a workflow orchestration platform. It helps data engineers schedule, organize, execute, monitor, and manage workflows.

Simple definition:

Airflow = Workflow Orchestration

Example data pipeline:

```
Customer Database
       ↓
   Extract Data
       ↓
   Validate Data
       ↓
 Transform Data
       ↓
   Load Data
       ↓
 Data Warehouse
```

Airflow does not mean that Airflow itself performs every transformation. Instead, Airflow coordinates jobs written in Python, SQL, Spark, shell scripts, dbt, cloud services, APIs, and other systems.

3. ETL and ELT Fundamentals

3.1 ETL

ETL means Extract → Transform → Load.

```
Source Database
      ↓
   Extract
      ↓
   Transform
      ↓
     Load
      ↓
Data Warehouse
```

Typical transformations include:

- Removing duplicates
- Handling missing values
- Changing data types
- Standardizing dates
- Joining datasets
- Calculating derived fields

3.2 ELT

ELT means Extract → Load → Transform.

```
Source Database
      ↓
   Extract
      ↓
      Load
      ↓
Data Lake / Data Warehouse
      ↓
   Transform
      ↓
Analytics / Curated Tables
```

Modern cloud platforms often support ELT because the destination platform can perform large-scale transformations efficiently.

4. What is Workflow Orchestration?

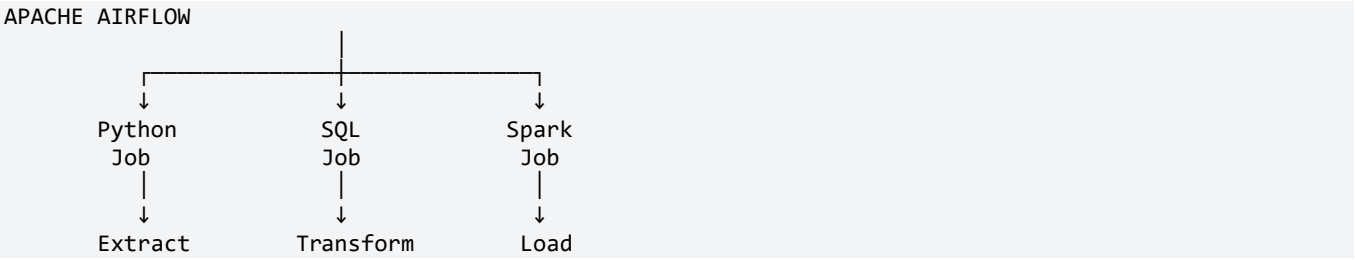
Workflow orchestration means coordinating multiple tasks, controlling their order, scheduling them, handling failures, and monitoring their execution.

```
Task A
  ↓
Task B
  ↓
Task C
```

If Task A fails, Task B should normally wait. Airflow can retry Task A and continue once it succeeds.

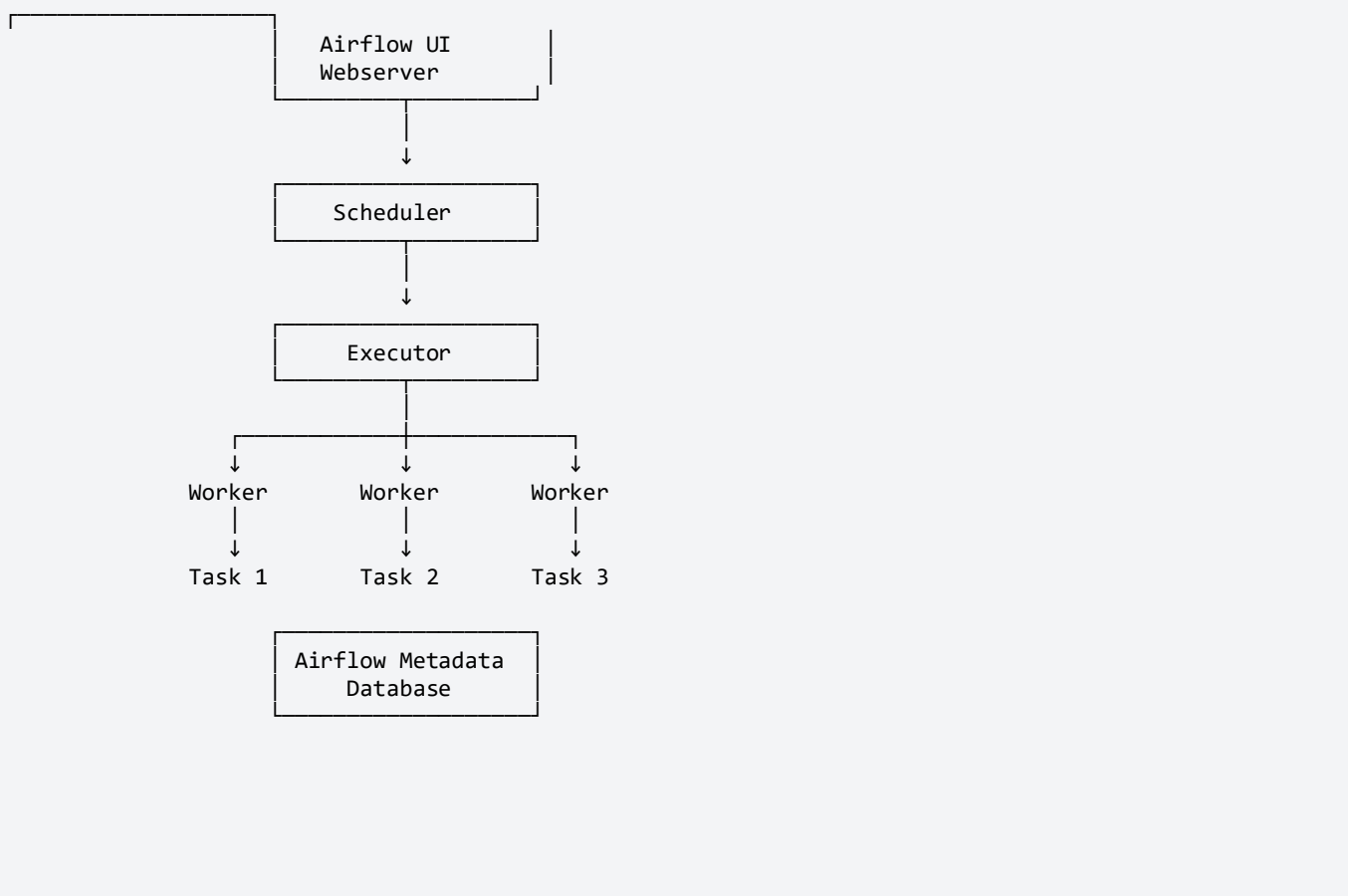
```
Task A
  ↓
FAILED
  ↓
Retry
  ↓
SUCCESS
  ↓
Task B
  ↓
Task C
```

5. Where Does Airflow Fit in a Data Pipeline?



The key lesson is: Airflow is primarily the orchestrator. The actual data processing may happen in Python, SQL, PySpark/Spark, cloud services, or other tools.

6. Apache Airflow Architecture



6.1 DAG

DAG stands for Directed Acyclic Graph. For a beginner, think of a DAG as the definition or blueprint of a workflow.

```
Extract
   ↓
Transform
   ↓
Load
```

Directed means tasks have a defined direction. Acyclic means the workflow does not contain an endless cycle.

6.2 Task

A Task is one unit of work, such as running a Python function, Bash command, SQL statement, or Spark job.

6.3 Operator

An Operator defines what a task should do.

Operator	Purpose
BashOperator	Execute shell/Bash commands
PythonOperator	Execute a Python function
SQL operators	Execute SQL work
Spark-related operators	Trigger Spark workloads
Email operators	Send notifications

6.4 Scheduler

The Scheduler determines which DAGs and tasks are due to run according to their schedule and dependencies.

6.5 Webserver / Airflow UI

The Webserver provides the browser-based Airflow UI where users can inspect DAGs, task states, logs, schedules, and execution history.

A common local development address is:

```
http://localhost:8080
```

6.6 Metadata Database

The metadata database stores Airflow's state and metadata, including DAG runs, task instances, task status, retries, scheduling information, connections, and variables.

6.7 Executor

The Executor controls how Airflow tasks are executed. The exact execution architecture depends on the executor selected for the deployment.

6.8 Worker

In worker-based architectures, workers provide the execution environment where task work actually runs.

7. DAG vs Task vs Operator

```
DAG
 │
 ├── Task 1 → PythonOperator
 │
 ├── Task 2 → BashOperator
 │
 └── Task 3 → PythonOperator
```

Simple mental model: DAG = workflow, Task = unit of work, Operator = how the work is performed.

8. Hands-on Example 1 — BashOperator

Goal: create a DAG that executes a Bash command.

```
from datetime import datetime

from airflow import DAG
from airflow.operators.bash import BashOperator


with DAG(
    dag_id="bash_operator_demo",
    start_date=datetime(2026, 10, 1),
    schedule="@daily",
    catchup=False,
) as dag:

    task1 = BashOperator(
        task_id="hello_bash",
        bash_command="echo 'Hello from Apache Airflow'"
    )
```

Architecture:

```
Airflow DAG
    ↓
BashOperator
    ↓
bash_command
    ↓
Operating System Shell
    ↓
Command Output
```

8.1 Practical Bash ETL-style DAG

```
from datetime import datetime

from airflow import DAG
from airflow.operators.bash import BashOperator


with DAG(
    dag_id="data_pipeline_bash_demo",
    start_date=datetime(2026, 10, 1),
    schedule="@daily",
    catchup=False,
) as dag:

    extract = BashOperator(
        task_id="extract_data",
        bash_command="echo 'Extracting customer data...'"
    )

    transform = BashOperator(
        task_id="transform_data",
        bash_command="echo 'Transforming customer data...'"
    )

    load = BashOperator(
        task_id="load_data",
        bash_command="echo 'Loading data into warehouse...'"
    )

    extract >> transform >> load
```

9. Hands-on Example 2 — PythonOperator

Goal: execute Python functions as Airflow tasks.

```
from datetime import datetime

from airflow import DAG
from airflow.operators.python import PythonOperator


def extract_data():
    print("Extracting customer data...")


def transform_data():
    print("Transforming customer data...")


def load_data():
    print("Loading customer data into warehouse...")


with DAG(
    dag_id="python_operator_demo",
    start_date=datetime(2026, 10, 1),
    schedule="@daily",
    catchup=False,
) as dag:
```

```
    extract = PythonOperator(
        task_id="extract_data",
        python_callable=extract_data
    )

    transform = PythonOperator(
        task_id="transform_data",
        python_callable=transform_data
    )

    load = PythonOperator(
        task_id="load_data",
        python_callable=load_data
    )

    extract >> transform >> load
```

9.1 PythonOperator Architecture

```
Airflow DAG
     ↓
PythonOperator
     ↓
Python Function
     ↓
Python Execution
     ↓
Result / Logs
```

10. Combined BashOperator + PythonOperator Example

This is a useful classroom example because it demonstrates that a single DAG can use different operators.

```
from datetime import datetime

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator


def validate_data():
    print("Validating customer data...")


def transform_data():
    print("Transforming customer data...")


with DAG(
    dag_id="mixed_operator_demo",
    start_date=datetime(2026, 10, 1),
    schedule="@daily",
    catchup=False,
) as dag:

    extract = BashOperator(
        task_id="extract_data",
        bash_command="echo 'Extracting customer data'"
    )

    validate = PythonOperator(
        task_id="validate_data",
        python_callable=validate_data
    )
```

```
    transform = PythonOperator(
        task_id="transform_data",
        python_callable=transform_data
    )

    load = BashOperator(
        task_id="load_data",
        bash_command="echo 'Loading data into warehouse'"
    )

    extract >> validate >> transform >> load
```

```
AIRFLOW DAG

   Extract
(BashOperator)
       ↓
   Validate
(PythonOperator)
       ↓
  Transform
(PythonOperator)
       ↓
     Load
(BashOperator)
```

11. BashOperator vs PythonOperator

Aspect	BashOperator	PythonOperator
Purpose	Execute shell/Bash command	Execute Python function
Main parameter	bash_command	python_callable
Good for	Shell scripts, CLI tools, OS commands	Python business/data logic
Example	spark-submit job.py	process_data()

12. Task Dependencies

The >> operator defines the execution order.

```
extract >> transform >> load
```

Meaning:

```
extract
   ↓
transform
   ↓
load
```

Branches can also run independently when their dependencies allow it:

```
┌── extract_customer ──→ transform_customer
│
Start ──────┤
│
└── extract_orders ────→ transform_orders
```

13. Scheduling

Airflow can schedule workflows at defined intervals.

Schedule	Meaning
@hourly	Hourly schedule
@daily	Daily schedule
@weekly	Weekly schedule
@monthly	Monthly schedule
0 2 * * *	Every day at 2:00 AM

14. Task States and Failure Handling

Common task states include:

- scheduled
- queued
- running
- success
- failed
- up_for_retry
- skipped
- upstream_failed

Retry example:

```
Load
 ↓
FAILED
 ↓
Retry
 ↓
FAILED
 ↓
Retry
 ↓
SUCCESS
```

15. DAG Run vs DAG

A DAG is the workflow definition or blueprint. A DAG Run is one actual execution of that workflow.

```
DAG
│
├── Run 1 → October 6
├── Run 2 → October 7
└── Run 3 → October 8
```

16. Task vs Task Instance

A Task is the definition. A Task Instance represents a particular execution of that task for a particular DAG run.

17. What to Demonstrate in the Airflow UI

- DAG list and DAG status
- Graph view showing task dependencies
- Grid view showing execution history
- Task details
- Task logs
- Task success/failure states
- Manual DAG trigger
- DAG run history

18. Real-World ETL + Airflow Architecture



19. Classroom Demo

Use a simple customer-data pipeline.

```
customers.csv
     ↓
Extract Task
     ↓
Validate Task
     ↓
Transform Task
     ↓
Load Task
     ↓
customer_final.csv / Data Warehouse
```

Recommended implementation:

- Use BashOperator for an extraction or shell command.
- Use PythonOperator for validation.
- Use PythonOperator for a simple transformation.
- Use BashOperator or another appropriate operator to demonstrate loading.
- Show the DAG in Graph view.
- Run the DAG manually.
- Open task logs and show success/failure.
- Intentionally make one task fail and demonstrate retry.

Revision

```
DAG         = Workflow definition
Task        = Unit of work
Operator    = How the task performs work
Scheduler   = Decides when tasks should run
Executor    = Determines how tasks are executed
Worker      = Execution environment in worker-based architectures
Webserver   = Airflow UI
Metadata DB= Stores Airflow state and metadata

BashOperator   → shell/Bash command
PythonOperator → Python function

ETL/ELT = Data processing approach
Airflow = Orchestrates the workflow
```

End of Reference